The Koselleck Machine

A project overview: what this instrument measures, who it is for, what it can and cannot support yet, and how to run or extend it.

REPOSITORY	koselleck-networks
DOCUMENT	docs/overview.tex → docs/overview.pdf
COMPANION	docs/method.pdf (the method and result in full); docs/pipeline_manual.pdf (internals)
STATUS	research instrument, local only, not yet deployed
COMPILED	September 9, 2026

Contents

1 What this is

The Koselleck Machine is a research instrument built to test one historical claim, and a small web tool for reading the result.

The claim is Reinhart Koselleck's. He argued that the decades between roughly 1770 and 1830, which he called the **Sattelzeit** (German for "saddle period"), were not merely a stretch of time in which many words happened to change meaning at once, but a moment when the vocabulary of politics and society reorganized **as a system**: concepts moved in relation to one another rather than drifting independently. That has always been argued in prose, from close reading. Ryan Heuser's *Computing Koselleck* was the first computational test of it: he trained a separate word-embedding model per historical period and measured how far each word moved from one period to the next, and found that movement is unusually large around 1770-1830. But a one-word-at-a-time measurement cannot distinguish a system reorganizing from a pile of unrelated shifts that happen to coincide in time. This project adds the missing test. It builds a word-similarity network for each period, runs community detection on it, and measures whether the **grouping structure itself** reorganizes at the Sattelzeit, rather than only measuring individual words.

One real example from the built data: in the 1770-1790 network, *nation* sits in a community of civic-administration words, while *revolution* sits in an unrelated community of shape and measurement terms. By 1790-1810, both words - along with *constitution* - have moved into a single new community of abstract systemic and governmental terms, exactly at the transition into the French Revolutionary period. `docs/method.pdf` covers this example, and the full result, in detail.

The full argument, the algorithms, and the exact measurements are set out in the companion document, `docs/method.pdf`. This overview does not repeat them. Its job is orientation: what the thing is, who it is for, what state it is in, and how to run it.

2 Who it is for, and what it is for right now

Two readers are in scope.

A historian or intellectual historian who wants to interrogate the result rather than take a summary statistic on faith. For that reader, the web tool is the product: type a word, see which semantic community it belonged to in every twenty-year window from 1510 onward, see explicitly where it moved and where it stayed, and drill into any period to read the actual nearest neighbours behind that judgement.

A computational or digital-humanities reader who wants to check the method, disagree with it, or run it on different text. For that reader, the product is the pipeline: seven standalone scripts, a single versioned configuration file, and a corpus contract small enough that substituting your own sources is a folder drop or one reader function (Section 5.4).

What it is not, yet. This is a research instrument, not a finished, citation-ready dataset. The numbers it produces are the object under test, not a published result; the community labels are a reading aid, not a validated taxonomy; and a full rerun does not reproduce an identical partition. Section 4 states each of those plainly. Treat anything read out of the tool as a lead to check, not as evidence to cite.

3 What is in the repository

`src/` is the pipeline, run in order: `parse_tcp.py` (raw sources to normalised per-document records), `bucket_periods.py` (records to 20-year text windows), `embeddings.py` (one word2vec model per window), `network.py` (each word linked to its 15 nearest neighbours by cosine similarity), `community.py` (Leiden community detection at fifteen resolution settings), and `metrics.py` (how much the grouping changed between consecutive periods: migration fraction (the share of shared words that ended up in a different community), plus NMI and adjusted Rand, two standard scores for how similar two groupings of the same words are). Alongside them sit `pipeline_config.py`, the shared configuration loader, and the two optional labeling scripts described below.

`webapp/` is the Koselleck Machine itself, a Flask application with a landing page and four views onto the same pre-built networks: `/timeline` (the primary view: one word's community across every period, in a single strip), `/chat` (**Ask**: a conversational front end that answers a plain-language question by calling tools that read the same built networks and metrics, never freehand generation, backed by `src/rag/`), `/graph` (a D3 force-directed explorer for one word's neighbourhood in one period, plus a sampled full-network mode), and `/search` (a plain lookup table: nearest neighbours, community, and whether the word's community changed since the previous period). Where the pipeline was also run per region, every page gains a region toggle, listing only the regions that were actually built.

`docs/` holds this overview and the method note.

`labels/` holds a small, citable snapshot of the current community labels, one CSV and one compiled JSON per region, copied there by `label_communities.py publish` so they travel with the code instead of living only in the gitignored data directory.

`config.yml` holds the settings that are shared and versioned: the period slices, and the word2vec and Leiden hyperparameters. Period boundaries and model parameters are configuration here, not code.

What the repository deliberately does *not* ship is the corpus or the trained models. That is a size decision, not a rights one: the raw Text Creation Partnership zips and the derived per-period networks run to many gigabytes, and the sources are public domain and fetchable directly. The contribution is the measurement, not the data.

4 Where it stands

Five limitations bound what can be claimed today. All five are stated in full in `docs/method.pdf` and surfaced inside the tool itself rather than hidden; they are summarised here without softening. (A separate, unrelated mechanism worth knowing about: the webapp displays and labels communities at a resolution picked automatically per period and region, kept small enough for a human to read - not a fixed value, and not what the resolution sweep below is tested against. `docs/method.pdf` covers it in full.)

Corpus coverage. The corpus is the Text Creation Partnership (EEBO-TCP, ECCO-TCP, Evans-TCP, curated and manually corrected, no OCR noise, 1500-1800), supplemented for 1800-1900 by the British Library's digitised 19th-century books collection (public domain, OCR-derived, catalogue metadata attached). A Project Gutenberg supplement was considered instead and dropped: its metadata carries only its own digitisation date, not the book's original publication date, which the method needs to bucket documents by period at all. The supplement is filed under the `british` region, alongside EEBO/ECCO - not because American and British English were different languages for most of this span (they were not, and treating the split as a dialect claim would not be defensible), but because it is a different, independently curated archive, so the region rerun tests whether the reorganization signal survives across two provenance sources rather than one, and the combined corpus stays the headline result. A direct consequence: the `american` region has no data past 1800 at all, since Evans-TCP stops there and the supplement is British-only. ECCO's own thinness after 1780 also still means the 1770-1790 window leans American-heavy even with the supplement in place, which only starts closing the gap in the window after it, 1790-1810. Per-region results carry wider uncertainty than the combined ones, and the American arm specifically cannot speak to anything after the Sattelzeit's closing edge.

OCR artifacts. Unlike TCP, the British Library supplement is OCR-derived, and OCR text carries its own artifacts. Two classes are repaired: an original line-break hyphen left sitting in the text (*"utrum- que"* for *utrumque*); and the harder case of the OCR dropping that hyphen entirely and leaving a bare space instead (*"par ticulars"* for *particulars*), fixed by merging two adjacent words only when the merged form is real and at least one half isn't independently real on its own - checked by hand against the one period a reference wordlist covers, 1790-1810 (142 merges made in the first 120 changed documents, all confirmed correct). That wordlist is built only from that period's own TCP text, so periods entirely after 1800 remain unrepaired and untested by this fix.

Word-level noise in the network. Two further patterns pass through into the trained networks and cluster tightly enough to form their own communities, neither yet fixed at the stage that would need to change (which word becomes a node): Latin citation-apparatus abbreviations (*capvt*, *deute*, *regu*) - correctly transcribed originals, not damage, but carrying no topical content either; and a distinct, unconfirmed TCP transcription pattern in a handful of communities from 1510-1650 - words missing a "con-"/"com-" prefix (*mytted* for *committed*, *uersation* for *conversation*), most likely a scribal abbreviation mark mishandled during extraction, not confirmed against the raw XML, and no fix attempted.

Label reliability. Every community carries a short plain-English name, generated by reading the community's most representative words. A label is a candidate to inherit from its predecessor only after two independent model reads, given the inherited label and the community's current words, agree it still fits; either reader saying no forces a fresh read instead. This replaced an earlier, blunter safeguard - a fixed cap on how many periods a label could inherit before a forced re-read - built after a real case: a community genuinely holding Dutch and German text at 1510-1530 was still labeled that way fifteen periods (roughly 300 years) later, by which point its content had passed through seven unrelated themes and landed on church governance, all unread. Checking the actual words on every candidate inheritance, rather than counting periods, catches that kind of drift directly instead of just bounding how long it can go unnoticed. Every fresh read, forced or genesis, is still a single, unvalidated pass - useful for orientation, not for citation. Separately, two-fifths to a third of communities in every region carry the "Structural / Uncertain" lane rather than a topical one, because the corpus is multilingual (English, Latin, Law French, Welsh, Scots, and more) and early modern, and a detected community sometimes groups words by shared grammar or shared language rather than by shared topic. The tool shows that lane next to the label rather than presenting every label as equally reliable.

Reproducibility. Rerunning the pipeline from scratch on the same corpus does not reproduce an identical result. Community detection is properly seeded and deterministic given a fixed network; the gap sits one step earlier, in training, which uses multiple parallel threads and is therefore not bit-for-bit repeatable even with a fixed seed, because thread scheduling varies. Small differences in the vectors cascade into slightly different edges and then into a structurally similar but not identical partition: two full reruns of the same combined corpus have produced community counts a few apart. That is a real difference, not a bug, and it is not worth trading away the threefold speed of parallel training to chase, because the claim under test is whether reorganization concentrates at the Sattelzeit and holds across the fifteen resolution settings within a run, not whether a specific community keeps its id across two independently retrained copies. The one thing designed to stay comparable across reruns is the fixed lane list, because it is authored by hand once and never reinvented.

There is also a standing methodological rule worth repeating here, because it governs everything the project will ever report:

Hard rule. *Resolution* is the one parameter that controls how many communities the detection step finds and how large they are - a higher resolution means more, smaller, more tightly-defined communities; a lower one means fewer, larger, looser ones - and it changes the answer. Every network is therefore processed at fifteen resolution settings, and no finding is reported if it holds only at a single, convenient one. The sweep exists to test that a conclusion survives across settings, not to pick the setting that agrees with the hypothesis.

Finally, on availability: the Koselleck Machine currently runs locally only. A public hosted deployment is intended but has not been started, deliberately kept out of scope until the local application is solid. The Flask app runs as-is on any host that can run Python, but it is not a static site and cannot be served from GitHub Pages, since every response is computed server-side from the network data.

5 Running it

5.1 Setup

```
python -m venv .venv
.venv/Scripts/activate  (or source .venv/bin/activate on macOS or Linux)
pip install -r requirements.txt
```

The pipeline expects a `data_root` folder containing `corpus/`, `processed/`, `embeddings/`, `networks/`, and `communities/`. Point at your own copy either by creating a gitignored `config.local.yml` with a `data_root:` entry, or by setting the `DATA_ROOT` environment variable, which takes priority over both config files. Only `corpus/` is filled in by hand; everything else is produced by the pipeline. The exact folder names come from `config.yml`'s `paths` block, which is authoritative if anything is renamed.

The corpus itself is fetched separately. All three TCP components used here are public domain, and the bulk P4 XML zip shards are downloaded directly from the Text Creation Partnership. They are laid out under `corpus/<region>/<source>/*.zip`, for example `british/eebo_phase1/`, `british/ecco/`, `american/evans/`. Both folder levels are free-form: `parse_tcp.py` reads the zips in place, without unpacking them, and discovers regions and sources by scanning that tree rather than from any hardcoded list. The top-level folder name becomes the region tag used throughout the pipeline and the webapp's region toggle. Only quality-checked "released" shards are read; TCP's "unedited" variants are skipped on purpose.

5.2 Running the pipeline

Each stage is a standalone script, run in order:

```
python src/parse_tcp.py
python src/bucket_periods.py
python src/embeddings.py
python src/network.py
python src/community.py
python src/metrics.py
```

That sequence is enough to reproduce every quantitative result. Community labels are a separate, optional stage on top of it, run as two more scripts:

```
python src/extract_community_words.py
python src/label_communities.py generate --region combined
python src/label_communities.py compile --region combined
python src/label_communities.py publish --region combined
```

`generate` produces a human-editable CSV, inheriting each community's label from its aligned predecessor wherever one exists and leaving only genuinely new communities blank for a person or an agent to fill in; `compile` turns the CSV into the JSON the webapp reads; and `publish` copies the CSV and JSON into the repository's `labels/` directory as a citable snapshot. `--region` also accepts a region name or `all`. The prompt the model is given lives in the repository as `src/prompts/community_labeling_v3.md`, and `docs/method.pdf` quotes and documents it in full.

5.3 Running the web tool locally

```
python webapp/app.py
```

Then open `http://127.0.0.1:5000`. Without real data the app still starts, but every period shows as a coverage gap. For a production host, the app is served with `gunicorn` and a `DATA_ROOT` environment variable pointing at wherever the network and community data lives on that machine; the data does not travel with the repository and has to be uploaded or fetched there separately.

5.4 Bringing your own corpus

The pipeline is built so that a reader can rerun the whole measurement on their own text rather than take these numbers on trust. There are exactly two cases, and only one of them touches code.

Case A: the material is already in TCP's P4 XML format (another TCP release, or a repackaged TCP subset). No code change at all. Drop the zip files at `<data_root>/corpus/<a-region-name>/<a-source-name>/*.zip` and rerun the pipeline from `src/parse_tcp.py`. Nothing needs registering anywhere: both folder names are free-form, the region level becomes the region tag carried through the pipeline and the webapp, the source level is a diagnostic label kept in the manifest, and `parse_tcp.py` discovers both by scanning.

Case B: the material is in any other raw format (plain text, JSON, a different XML flavour, OCR dumps). One file changes: `src/parse_tcp.py`. Add a reader next to the existing `iter_xml_members` that walks the new format and, for each document, emits the same record the TCP path already writes to `processed/all_docs.jsonl`:

```
{"region": "...", "source": "...", "doc_id": "...", "year": 1782, "text": "..."}
```

plus the matching `region`, `source`, `doc_id`, `year`, `chars` row in `manifest.csv`. Documents with no extractable year or no text are dropped, as in the TCP path. That is the entire contract. Everything downstream, `bucket_periods.py` through `metrics.py` and the webapp, reads only those normalised records and needs no change; period boundaries and model hyperparameters are configuration in `config.yml`, not code. In both cases, a region name that did not exist before is picked up automatically by the region-split outputs and the webapp's region toggle; there is no list of valid regions to update.

To be explicit about what does not exist: there is no upload interface, no hosted ingestion service, and no API-key-driven pipeline. The mechanism is the ordinary one, clone the repository, put your corpus files in the right folder, rerun the scripts in order.

6 Where to read more

`docs/method.pdf` is the full method note: the argument, the algorithms, the resolution sweep and why it is non-negotiable, the actual result and what it does and does not show for Koselleck's claim, the labeling prompt and workflow in full, and the limitations in their unabridged form. `docs/pipeline_manual.pdf` covers the implementation stage by stage, for a reader who wants to see exactly what each script does or is troubleshooting a specific one. The repository's `README.md` carries the operational detail: corpus download links and licensing, the exact expected data layout, and deployment notes.